



1. Зачем агенту нужен проектный манифест

На прошлом занятии мы работали с Planner, Coder и Reviewer и каждый раз достаточно подробно объясняли субагенту, что он должен делать, с каким проектом работает и какие ограничения необходимо соблюдать. Теперь возникает логичный вопрос: что делать, если одни и те же правила должны действовать постоянно?

Представьте большой проект, в котором каждому новому агенту приходится заново объяснять, какой язык используется, где находятся тесты, какие файлы нельзя изменять и какой командой нужно проверять результат. Это не только неудобно, но и создаёт риск расхождения инструкций. В одном запросе мы можем написать одно правило, в другом забыть его указать, а в третьем сформулировать немного иначе.

Именно для этого проекту нужен постоянный контекст. В Hermes одним из основных способов задать такой контекст является файл `AGENTS.md`. Я буду условно называть его манифестом проекта для агента. В нём мы фиксируем не конкретную одноразовую задачу, а правила, которые должны учитываться при работе с этим проектом вообще. Hermes автоматически загружает такой файл в контекст агента.

2. Инициализация агента и начальный контекст

Когда агент начинает новую работу, он не должен каждый раз самостоятельно угадывать устройство проекта. Нам желательно заранее дать ему исходное понимание рабочей среды.

Инициализацию в данном случае я рассматриваю как формирование начальных условий работы агента. Модель должна получить информацию о том, с каким проектом она взаимодействует, какие соглашения здесь действуют и какие действия считаются допустимыми.

В Hermes проектный контекст загружается в системную часть запроса при старте сессии. `AGENTS.md` поэтому отличается от обычного сообщения пользователя. Сообщение относится к конкретному заданию, а содержимое проектного контекста сопровождает работу агента и влияет на последующие действия внутри проекта.

Например, конкретная задача может звучать как «проверь миграцию зависимости». А проектный контекст уже заранее сообщает, что тесты запускаются определённой командой, миграционные файлы нельзя изменять вручную и публичные интерфейсы необходимо сохранять совместимыми.

3. Что именно представляет собой AGENTS.md в Hermes

Важно не воспринимать `AGENTS.md` как специальный исполняемый конфигурационный файл. Это обычный Markdown-документ, который содержит понятные для модели текстовые инструкции.

То есть внутри него мы описываем проект естественным языком. Там может быть указано назначение проекта, его архитектура, соглашения по коду, команды проверки, важные каталоги и ограничения.

Hermes обнаруживает этот файл автоматически и добавляет его содержимое в проектный контекст. В официальной документации `AGENTS.md` определяется как основной файл для описания структуры проекта, соглашений, архитектуры и специальных инструкций.

Это важно с архитектурной точки зрения. Мы не программируем поведение агента в традиционном смысле. Мы создаём декларативное описание того, в каких условиях агент должен принимать решения.

4. Цель агента и цель проекта

В манифесте полезно разделять постоянную цель роли и конкретную задачу текущего запуска.

Например, если у нас есть агент-ревьюер, его постоянная роль может заключаться в проверке изменений без самостоятельного исправления кода. Но конкретная задача сегодня может быть связана с миграцией Pydantic, завтра с обновлением версии API, а послезавтра с проверкой изменения схемы базы данных.

Поэтому я не хочу превращать `AGENTS.md` в огромный набор конкретных заданий. Его задача заключается в фиксации устойчивых правил.

Для будущего reviewer мы можем концептуально определить, что его основная ответственность заключается в независимой проверке изменений. Для migrator можно установить, что его задача связана с адаптацией существующего проекта при сохранении внешнего поведения.

А вот точное название зависимости, текущая версия и конкретный объём миграции уже относятся к задаче текущего запуска.

5. Роль агента и границы ответственности

Теперь появляется понятие роли. Мы уже использовали роли Planner, Coder и Reviewer, но раньше определяли их непосредственно в запросах. Через проектный контекст можно сделать такое разделение более устойчивым.

Роль отвечает на вопрос: за какую часть процесса агент несёт ответственность.

Если агент выступает Reviewer, ему не обязательно запрещать вообще взаимодействовать с проектом. Ему может потребоваться читать файлы, смотреть Git diff и запускать тесты. Но изменение производственного кода уже может находиться за границей его ответственности.

Для Migrator ситуация другая. Он, наоборот, должен иметь возможность менять файлы, потому что это является основной частью его работы. Но даже ему не обязательно разрешать изменение любых компонентов проекта.

То есть хорошая роль определяется не красивым названием, а конкретными границами. Слова `Reviewer` или `Migrator` сами по себе ничего технически не гарантируют. Мы должны объяснить модели, что считается допустимым поведением внутри этой роли.

6. Навыки агента

Следующая составляющая структуры агента — навыки. Здесь я хочу провести важное различие между ролью и навыком.

Роль отвечает за область ответственности, а навык описывает то, что агент должен уметь делать для выполнения этой ответственности.

Reviewer может уметь анализировать Git diff, читать тесты, сопоставлять изменения с требованиями и находить потенциальные нарушения обратной совместимости.

Migrator должен понимать конфигурацию зависимостей, находить использование устаревшего API, адаптировать исходный код и проверять сохранение поведения после обновления.

При этом описание навыка ещё не означает, что агент автоматически получает соответствующий технический инструмент. Мы можем написать, что агент должен анализировать репозиторий, но без возможности читать файлы он физически не сможет полноценно выполнить такую работу.

Поэтому навыки необходимо рассматривать вместе с инструментами.

7. Инструменты агента

Инструмент — это уже не описание интеллектуальной способности, а конкретная техническая возможность взаимодействовать с окружающей средой.

В Hermes агент может работать с файлами, терминалом, поиском по проекту и другими инструментами. В предыдущей практике мы видели, что именно инструменты превращают языковую модель из системы, которая просто формирует текстовый ответ, в систему, способную реально исследовать проект и выполнять действия.

В `AGENTS.md` мы можем описать, какие инструменты предполагается использовать и какие действия считаются нормальными для проекта. Но здесь важно понимать границу: текстовая инструкция и техническое разрешение — не одно и то же.

Если в манифесте написано «не изменяй конфигурацию production», это правило поведения. Но более надёжная архитектура дополнительно ограничивает реальные доступные возможности агента там, где это возможно.

Поэтому правила в контексте помогают модели принимать правильные решения, а технические разрешения обеспечивают дополнительную границу исполнения.

8. Лимиты и ограничения

Теперь переходим к одной из самых важных частей манифеста — ограничениям.

Часто хороший `AGENTS.md` определяется не столько тем, что агенту разрешено делать, сколько тем, что ему делать нельзя.

Например, проект может разрешать обновление зависимостей, но запрещать автоматическое изменение схемы базы данных. Можно разрешить запуск тестов, но запретить отправку изменений в удалённый репозиторий. Можно позволить работать с исходным кодом, но не разрешать читать файлы с секретами.

Ограничения нужны ещё и для защиты от чрезмерной инициативы агента. Модель может увидеть возможность улучшить соседний модуль и решить, что дополнительный рефакторинг будет полезен. Но если задача заключалась только в миграции одной зависимости, такие изменения увеличивают область риска.

Поэтому лимит — это не наказание для агента. Это способ удержать выполнение внутри понятной области ответственности.

9. Правила безопасности

Правила безопасности я рассматриваю шире, чем просто запрет на удаление файлов.

В агентной разработке опасность часто возникает из комбинации нескольких вроде бы нормальных действий. Агент умеет читать файлы, выполнять команды терминала и работать с Git. Каждая возможность сама по себе полезна, но вместе они уже позволяют существенно воздействовать на проект.

Поэтому в проектном контексте необходимо заранее определять чувствительные области. Это могут быть секреты, production-конфигурация, миграции базы данных, инфраструктурные скрипты или другие компоненты, где ошибка особенно дорого обходится.

Hermes дополнительно проверяет загружаемые контекстные файлы на потенциально опасные инструкции перед добавлением их в контекст, а проектные файлы проходят такую проверку как при первоначальной загрузке, так и при последующем обнаружении внутри подкаталогов.

Я хочу, чтобы мы воспринимали безопасность не как отдельный финальный пункт после разработки агента. Она должна быть частью самой структуры его роли.

10. AGENTS.md как иерархия правил проекта

Последняя особенность особенно важна для больших репозиториях. В Hermes `AGENTS.md` может существовать не только в корневом каталоге.

Представим проект, где backend и frontend живут в одном репозитории. Общие правила можно определить в корневом `AGENTS.md`, а внутри отдельных каталогов добавить более специфичные инструкции.

Hermes умеет загружать цепочку таких файлов от корня Git-репозитория к рабочему каталогу. Более глубокие инструкции появляются позже и поэтому могут уточнять правила конкретной части проекта. Кроме того, при переходе агента в новые подкаталоги Hermes способен обнаруживать соответствующий `AGENTS.md` уже во время работы.

Это позволяет не создавать один огромный универсальный манифест. Общие правила остаются на уровне проекта, а специфические требования размещаются ближе к соответствующей части кодовой базы.

Для нашей дальнейшей работы это особенно полезно. Мы сможем рассматривать Reviewer и Migrator не как агентов, которым каждый раз вручную передают огромную инструкцию, а как исполнителей, работающих внутри заранее определённого проектного контекста.

При этом важно знать ещё одну особенность Hermes. Помимо `AGENTS.md`, существует Hermes-специфичный файл `.hermes.md` или `HERMES.md`, который имеет более высокий приоритет. Hermes выбирает проектный контекст по установленному порядку приоритетов, поэтому одновременное наличие нескольких типов файлов не означает, что все они обязательно будут объединены.

В результате я хочу, чтобы после этой темы мы воспринимали структуру агента как сочетание нескольких вещей. У него есть назначение, область ответственности, необходимые способности, доступные действия и ограничения. `AGENTS.md` позволяет сделать значительную часть этих правил постоянной частью проекта, чтобы агент начинал работу уже с пониманием того, в какой среде он находится и какие требования должен соблюдать.
